

Faculty of Engineering and Technology

Electrical and Computer Engineering Department

APPLIED CRYPTOGRAPHY

ENCS4320

Report Project

A Secure Communication Application

Prepared by:

Student's Name: Ghazal Salah **Student's Id: 1230542**

Partner's Name: Baker Shalabi **Partner's Id: 1231050**

Repository:

https://github.com/ghazaldaraghmeh-lang/SecureChannel-A-Secure-Communication-Application

Instructor: Dr. Ahmad Shawahna

Section: 1

Date: 27/06/2026

Table of Contents

1. Introduction ---------- 1
2. System Architecture---------- 1
   - 2.1 Layers---------- 1
   - 2.2 File Structure---------- 2
3. Cryptographic Primitives---------- 2
4. Protocol Design---------- 2
   - 4.1 Phase A – Handshake---------- 3
   - 4.2 Phase B – Secure Messaging---------- 3
5. Message Format---------- 3
   - 5.1 Application Message Structure---------- 3
     - Header (AAD) ---------- 3
     - Ciphertext---------- 3
     - Authentication Tag ---------- 3
6. Key Schedule ---------- 4
7. Authentication Mechanism ---------- 4
8. Nonce Construction---------- 4
9. Security Analysis----------5
   - 9.1 Security Properties----------5
   - 9.2 Threat Resistance---------- 5
   - 9.3 Security Summary ---------- 5
   - 9.4 Limitations ---------- 5
10. Replay Protection----------6
11. Tampering Protection ----------6
12. Testing Results---------- 6
13. References----------7
14. AI Usage Statement ---------- 7
15. Individual Contribution Statements----------7
16. Conclusion ---------- 7

➢ Introduction

This project presents the design and implementation of a secure end-to-end communication system in Python. The system enables two parties (client and server) to communicate over an insecure network while ensuring confidentiality, integrity, authentication, and replay protection (CIA properties).

The protocol is inspired by modern secure communication systems such as TLS 1.3 and is structured into two main phases:

- **Phase A (Handshake):** Mutual authentication and shared secret establishment using X25519 ephemeral key exchange combined with HMAC-based authentication.
- **Phase B (Secure Messaging):** Confidential and authenticated communication using AEAD encryption (ChaCha20-Poly1305).

All cryptographic primitives are implemented from scratch according to official standards, including:

- SHA-256 (FIPS 180-4)
- HMAC (RFC 2104 / RFC 4231)
- HKDF (RFC 5869)
- X25519 (RFC 7748)
- ChaCha20-Poly1305 (RFC 8439)

➢ System Architecture

The system follows a modular layered architecture similar to TLS 1.3.

■ Layers

- **Transport Layer:** Handles TCP socket communication, packet framing, and sequence tracking.
- **Handshake Layer:** Manages key exchange and authentication logic.
- **Cryptographic Layer:** Implements hashing, encryption, MAC, and key derivation functions.
- **Utility Layer:** Handles serialization, nonce construction, and message formatting.

■ File Structure

- client.py, server.py → Communication endpoints
- handshake.py → Key exchange and authentication
- aead.py, chacha20.py, poly1305.py → Encryption and authentication
- x25519.py, hkdf.py → Key exchange and derivation
- sha256.py, hmac.py → Hashing and MAC

➢ 3. Cryptographic Primitives

All primitives are implemented based on official standards without external cryptographic libraries.

- **SHA-256 (FIPS 180-4):** Cryptographic hash function producing a 256-bit digest using 64 compression rounds.
- **HMAC (RFC 2104):** Message authentication using a secret key:
  $H(K \oplus opad \parallel H(K \oplus ipad \parallel M))$
- **HKDF (RFC 5869):** Key derivation mechanism with Extract and Expand phases.
- **X25519 (RFC 7748):** Elliptic Curve Diffie-Hellman using Montgomery ladder for constant-time operations.
- **ChaCha20 (RFC 8439):** Stream cipher based on ARX operations over 20 rounds.
- **Poly1305 (RFC 8439):** One-time message authentication code using polynomial evaluation.
- **ChaCha20-Poly1305 AEAD:** Combines encryption (ChaCha20) and integrity (Poly1305).

➢ 4. Protocol Design

- **Phase A – Handshake**

1. Both client and server generate ephemeral X25519 key pairs.
2. Public keys are exchanged over the network.
3. A shared secret is computed using ECDH:
   shared_secret = X25519(private_key, peer_public_key)

4. The handshake transcript is authenticated using HMAC-SHA256 with a pre-shared key (PSK).
5. Session keys are derived using HKDF-SHA256.

   If authentication fails, the connection is immediately terminated.

   - **Phase B – Secure Messaging**

   After handshake completion, secure messaging begins:

- Messages are encrypted using **ChaCha20-Poly1305 AEAD**
- Each message includes:
  - Sequence number
  - Sender identity
  - Protocol version

  Replay protection is enforced through strict sequence tracking.

➢ Message Format

■ Application Message Structure

Each encrypted message in the system follows the structure:

Header (AAD) || Ciphertext || Poly1305 Tag

Header (Associated Data - AAD)

The header is transmitted in plaintext but authenticated. It includes:

- Protocol version
- Message type (handshake or data)
- Sender ID
- Sequence number

Ciphertext

The encrypted payload generated using the ChaCha20 stream cipher.

Authentication Tag

Generated using Poly1305 as part of the ChaCha20-Poly1305 AEAD construction.

The header is not encrypted but is fully integrity-protected.

➢ Key Schedule

Session keys are derived using HKDF from the shared secret:

- Client → Server encryption key
- Server → Client encryption key
- Base nonce seed

The PSK is used as HKDF salt to bind the session context to authentication.

➢ Authentication

A pre-shared key (PSK) is used to authenticate the handshake:

HMAC(PSK, handshake_transcript)

The handshake transcript includes:

- Client public key
- Server public key
- Identity information
- Protocol version

If authentication fails, the connection is terminated.

➢ Nonce Construction

Each message uses a unique nonce to ensure AEAD security:

Nonce = 12-byte base nonce ⊕ sequence number

This guarantees:

- No nonce reuse
- Protection against replay and reuse attacks
- Safe ChaCha20-Poly1305 encryption

➢ Security Analysis

■ Security Properties

- Confidentiality → ChaCha20 encryption
- Integrity → Poly1305 MAC
- Authentication → PSK-based handshake
- Forward secrecy → Ephemeral X25519 keys
- Replay protection → Sequence numbers

■ Threat Resistance

- MITM attacks: Prevented via handshake transcript authentication
- Replay attacks: Blocked via sequence tracking
- Tampering: Detected via AEAD verification
- Nonce reuse: Prevented via HKDF + sequence-based nonce

Security Summary

The protocol provides:

- Confidentiality → ChaCha20 encryption
- Integrity → Poly1305 MAC
- Authentication → HMAC-based handshake
- Secure key exchange → X25519
- Key derivation → HKDF-SHA256
- Replay protection → sequence numbers

■ Limitations

- No constant-time comparisons (possible timing leakage)
- Strict in-order message delivery only
- No message buffering or reordering support

➢ Replay Protection

Replay attacks are prevented using sequence numbers:

- Each message includes a sequence number in the header
- The receiver tracks the expected sequence number
- Messages with duplicate or old sequence numbers are rejected

This ensures that replayed messages cannot be accepted even if ciphertext and authentication tag are valid.

➢ Tampering Protection

Tampering is prevented using ChaCha20-Poly1305 AEAD:

- Any modification in ciphertext causes decryption failure
- Any modification in AAD (header) causes authentication failure

This ensures both confidentiality and integrity of messages.

➢ Testing

All primitives were validated against official RFC/FIPS test vectors:

Primitive	Standard	Result
SHA-256	**FIPS 180-4**	**PASS**
HMAC-SHA256	**RFC 2104 / 4231**	**PASS**
HKDF	**RFC 5869**	**PASS**
ChaCha20	**RFC 8439**	**PASS**
Poly1305	**RFC 8439**	**PASS**
AEAD	**RFC 8439**	**PASS**
X25519	**RFC 7748**	**PASS**

➢ References

- SHA-256 (FIPS 180-4):
- https://csrc.nist.gov/pubs/fips/180-4/final
- HMAC (RFC 2104):
- https://datatracker.ietf.org/doc/html/rfc2104
- HKDF (RFC 5869):
- https://datatracker.ietf.org/doc/html/rfc5869
- ChaCha20 / Poly1305 / AEAD (RFC 8439):
- https://datatracker.ietf.org/doc/html/rfc8439
- X25519 (RFC 7748):
- https://datatracker.ietf.org/doc/html/rfc7748

➢ AI Usage Statement

- **AI tools were used only for:**

- Explaining cryptographic concepts
- Debugging implementation issues
- Clarifying RFC documentation

No AI tool was used to implement core cryptographic algorithms or protocol logic. All primitives and protocol design were implemented manually.

➢ Individual Contribution Statements

The tasks were distributed among the team members, and all members collaborated in a coordinated and organized manner to complete the required work. This teamwork contributed to the successful completion of the project within the specified time.

➢ Conclusion

This project demonstrates a complete secure communication system built entirely from cryptographic primitives implemented from scratch.

All cryptographic components follow official standards (RFC 7748, RFC 5869, RFC 8439, and FIPS 180-4) without using external cryptographic libraries.

The system successfully ensures secure key exchange, authenticated encryption, and protection against real-world attacks such as replay and tampering.